26 giugno 2019



Politecnico di Milano

Anno accademico 2018-2019

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Margara	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica della classe `RecentContacts` che modella la lista dei contatti di una chat ordinati sulla base dell'ultima interazione (dalla più alla meno recente).

```
public class RecentContacts {
  // Crea una lista di contatti vuota.
  public RecentContacts();

  // Aggiunge un nuovo contatto in fondo alla lista.
  // Lancia l'eccezione ExistingContactException
  // se il contatto è già in lista.
  public void add(String contact) throws ExistingContactException;

  // Rimuove il contatto dalla lista.
  // Lancia l'eccezione UnknownContactException se il contatto non è in lista.
  public void remove(String contact) throws UnknownContactException;

  // Metodo invocato quando avviene un'interazione
  // (invio o ricezione di un messaggio) con il contatto contact.
  // Lancia l'eccezione UnknownContactException se il contatto non è in lista.
  public void notifyInteraction(String contact) throws UnknownContactException;

  // Restituisce il contatto in posizione pos.
  public /*@ pure @*/ String get(int pos);

  // Restituisce il numero di contatti presenti in lista.
  public /*@ pure @*/ int size();
}
```

Domanda a)

Si specifichino in JML i metodi `add`, `remove` e `notifyInteraction`.

Soluzione

```
//@requires contact != null
//@ensures \forall(int i; 0<=i<\old(size()); !contact.equals(\old(get(i)))) &&
//@        \forall(int i; 0<=i<\old(size()); get(i).equals(\old(get(i)))) &&
//@        get(size()-1).equals(contact) &&
//@        size() == \old(size())+1
//@signals(ExistingContactException)
//@        \exists(int i; 0<=i<\old(size()); contact.equals(\old(get(i)))) &&
//@        \forall(int i; 0<=i<\old(size()); get(i).equals(\old(get(i)))) &&
//@        size()==\old(size())
public void add(String contact) throws ExistingContactException;

//@requires contact != null
//@ensures \exists(int i; 0<=i<\old(size());
//@            contact.equals(\old(get(i))) &&
//@            \forall(int j; 0<=j<i; get(j).equals(\old(get(j)))) &&
//@            \forall(int j; i<=j<size(); get(j).equals(\old(get(j+1))))) &&
//@        size() == \old(size())-1
//@signals(UnknownContactException)
//@        \forall(int i; 0<=i<\old(size()); !contact.equals(\old(get(i)))) &&
//@        \forall(int i; 0<=i<\old(size()); get(i).equals(\old(get(i)))) &&
//@        size()==\old(size())
public void remove(String contact) throws UnknownContactException;
```

```
//@requires contact != null
//@ensures \exists(int i; 0<=i<\old(size());
//@            contact.equals(\old(get(i))) &&
//@            get(0).equals(contact) &&
//@            \forall(int j; 1<=j<=i; get(j).equals(\old(get(j-1)))) &&
//@            \forall(int j; i<j<size(); get(j).equals(\old(get(j))))) &&
//@        size() == \old(size())
//@signals(UnknownContactException)
//@        \forall(int i; 0<=i<\old(size()); !contact.equals(\old(get(i)))) &&
//@        \forall(int i; 0<=i<\old(size()); get(i).equals(\old(get(i)))) &&
//@        size()==\old(size())
public void notifyInteraction(String contact) throws UnknownContactException;
```

Domanda b)

Si specifichi in JML l'invariante pubblico del tipo `RecentContacts`. Esistono proprietà rilevanti per la classe che non si possono esprimere con un invariante?

Soluzione

```
//@public invariant
//@ size() >=0 &&
//@ \forall(int i; 0<=i<size(); get(i)!=null) &&
//@ \forall(int i; 0<=i<size();
//@   \forall(int j; 0<=j<size() && i!=j; !get(i).equals(get(j))))
```

Non è possibile esprimere proprietà evolutive, in particolare non si può affermare che l'ordine con cui si trovano i contatti riflette l'ordine di invocazione dei metodi `add`, `remove` e `notifyInteraction`. Tale proprietà viene comunque garantita dalle post-condizioni dei modificatori.

Domanda c)

Si consideri il tipo `Contacts`, che mantiene la lista dei contatti senza un ordine preciso (le postcondizioni dei modificatori non specificano un ordine preciso per gli elementi). Il tipo `Contacts` può essere sottotipo di `RecentContacts` secondo il principio di sostituibilità di Liskov? È possibile il contrario? Motivare le proprie risposte.

Soluzione

Il tipo `Contacts` non può essere sottotipo di `RecentContacts` perché non rispetterebbe il vincolo di ordinamento. Ad esempio, questo indebolirebbe le post-condizioni di `add` e `notifyInteraction`.

Il tipo `RecentContacts` può invece essere sottotipo di `Contacts`, in quanto l'ordine cronologico di interazione rappresenta un caso particolare dell'assenza di ordine (ovvero di ordinamento qualsiasi, non deterministico).

Esercizio 2

Si consideri la seguente classe `LongSequence` che realizza una sequenza di valori `long` positivi. Ogni elemento della sequenza è inizialmente "vuoto", ovvero posto a `-1`. I metodi `get` e `set` consentono di leggere (rimuovendo) o scrivere un valore in una certa posizione della sequenza, sospendendo il chiamante se la posizione è vuota (o piena nel caso del `set`). Il metodo `getSize` restituisce la capacità della sequenza (ovvero il numero di elementi che può contenere).

```
public class LongSequence {
    private long[] data;
    public LongSequence(int size) {
```

```
        data = new long[size];
        for(int i=0; i<size; i++) data[i]=-1;
    }
    public long get(int pos)  throws InterruptedException {
        long result;
        synchronized(data) {
            while(data[pos]<0) wait();
            result = data[pos];
            data[pos] = -1;
            notifyAll();
        }
        return result;
    }
    public void set(int pos, long val) throws InterruptedException {
        synchronized(data) {
            while(data[pos]>=0) wait();
            data[pos] = val;
            notifyAll();
        }
    }
    public int getSize() {
        return data.length;
    }
}
```

Domanda a)

La sincronizzazione è corretta per tutti i metodi? In caso affermativo motivare la risposta, in caso negativo indicare come andrebbe modificato il codice per avere una sincronizzazione corretta.

Soluzione

I metodi `get` e `set` correttamente acquisiscono il lock sull'oggetto `data` prima di modificarlo, ma invocano la `wait` e `notifyAll` su `this` invece che su `data` stesso. Occorre quindi cambiare le istruzioni `wait()` in `data.wait()` e analogamente cambiare `notifyAll()` in `data.notifyAll()`.

Si noti che il metodo `getSize` non richiede sincronizzazione perchè la lunghezza dell'array `data` è un valore immutabile.

Domanda b)

Si aggiunga il metodo `public void setAllAsync(long val)` alla classe `LongSequence`. Il metodo deve impostare tutti i valori della sequenza a `val` indipendentemente dal precedente stato dell'elemento (pieno o vuoto) e deve farlo in maniera asincrona rispetto al chiamante (cioè usando un thread separato). Si usino solo i meccanismi base della sincronizzazione di Java senza sfruttare classi di libreria.

Soluzione

```
    public void setAllAsync(long val) {
        new Thread() {
            public void run() {
                synchronized(data) {
                    for(int i=0; i<data.length; i++) data[i]=val;
                    data.notifyAll();
                }
```

```
            }
        }.start();
    }
```

Esercizio 3

Si consideri il metodo `findMyIntegers`, non completamente definito, della seguente classe `Exam`:

```
public class Exam {
   public static List<Integer> findMyIntegers(List<Integer> aList, Predicate<Integer> aPredicate) {
      BiFunction<List<Integer>, Predicate<Integer>, List<Integer>> onlyMine;
      onlyMine = (myListOfInt, myPredicate) -> {
         ........................................
         ........................................
      };
      return onlyMine.apply(aList, aPredicate);
   }

   public static void main(String[] s){
      Predicate<Integer> predicate = x -> x % 2 == 0;
      List<Integer> startingList = Arrays.asList(1, 7, 4, 6, 8, 9);
      List<Integer> finalList = findMyIntegers(startingList, predicate);
      System.out.println(finalList);
   }
}
```

Il metodo applica il predicato `aPredicate`, definito con l'interfaccia funzionale `Predicate<Integer>`, al parametro `aList`, restituendone, come una nuova lista, tutti e soli i valori che soddisfano il predicato stesso.

A tale scopo, il codice utilizza l'interfaccia funzionale `Bifunction` per definire una funzione che riceve come argomenti una lista e un `Predicate` di `Integer`, restitutendo una lista.

La definizione delle due interfacce funzionali viene ricordata di seguito:

```
@FunctionalInterface
public interface BiFunction<T,U,R> {
   // Apply this function to two arguments;
   R apply(T t, U u);
}

@FunctionalInterface
public interface Predicate<T> {
   // Evaluates this predicate on the given argument:
   boolean test(T t);
   ...
}
```

Completare il codice del metodo usando preferibilmente un approccio funzionale. Indicare inoltre il risultato dell'esecuzione del metodo `main` interno alla classe `Exam`.

Soluzione

Basta inserire al posto puntini la seguente riga:

```
return myListOfInt.stream().filter(myPredicate).collect(Collectors.toList());
```

Il main stampa 4, 6, 8.

Esercizio 4

Si consideri il seguente metodo Java:

```
public static int m(int arr[]) {
  int sum = 0;
  if (arr == null || arr.length == 0) {
    sum = -1;
  } else {
    for (int i = 0; i < arr.length % 2; i++) {
      if (arr[i] == i) {
        sum += i;
      }
    }
  }
  return sum;
}
```

Domanda a)

Si fornisca un insieme di casi di test per la copertura delle decisioni (branch/edge coverage).

Soluzione

Occorre un caso in cui si entra nel primo ramo `if` (array nullo o vuoto). Occorre valutare positivamente e negativamente la condizione del ciclo `for` (basterebbe un caso). Occorre valutare positivamente e negativamente la condizione del secondo `if` (servono due casi separati in quanto nel ciclo `for` si entra al massimo una volta).

```
arr = null,
arr = [0],  // Entra una volta nel for ed esegue if interno
arr = [1]   // Entra una volta nel for e non esegue if interno
```

Domanda b)

Si fornisca un insieme di casi di test per la copertura dei cammini.

Soluzione

In caso di array non nullo e non vuoto, il codice entra nel ciclo `for` nessuna volta (quando arr ha dimensione pari) o una volta (quando array ha dimensione dispari). Per coprire tutti i cammini occorre quindi avere un caso di test che non entri mai nel ciclo, uno che ci entri una volta ed esegua il ramo `if` e uno che ci entri una volta e non esegua il ramo `if`. In aggiunta serve un caso con array nullo oppure vuoto per coprire il cammino in cui si entra nel primo `if`.

```
arr = null,
arr = [1, 1],  // Non entra nel ciclo for
arr = [0],     // Entra nel ciclo for una volta e nell'if
arr = [1]      // Entra nel ciclo for una volta ma non nell'if
```